

Public Transport Telemetry & Weather Impact Pipeline

Live Demo

Dashboard (Render): <https://transport-telemetry-dashboard.onrender.com>

The dashboard reads precomputed parquet outputs and reflects recent pipeline runs.

Tip: Select a route and toggle weather context in the Map View to explore how external signals relate to transport behavior.

A lightweight data engineering project focused on reliable data integration, simple aggregation logic, and practical monitoring outputs.

This project combines operational telemetry signals with external weather data into a unified pipeline, producing stable, queryable metrics for analysis and dashboards.

The focus is not on complexity, but on clarity, maintainability, and realistic engineering decisions.

Overview

Public transport systems are influenced by external conditions such as weather, but these signals are often processed separately.

This project demonstrates how multiple data sources can be integrated into a single pipeline and transformed into usable operational metrics.

Key goals:

- integrate heterogeneous data sources
 - apply consistent aggregation logic
 - produce stable outputs for downstream use
 - keep the system simple and maintainable
-

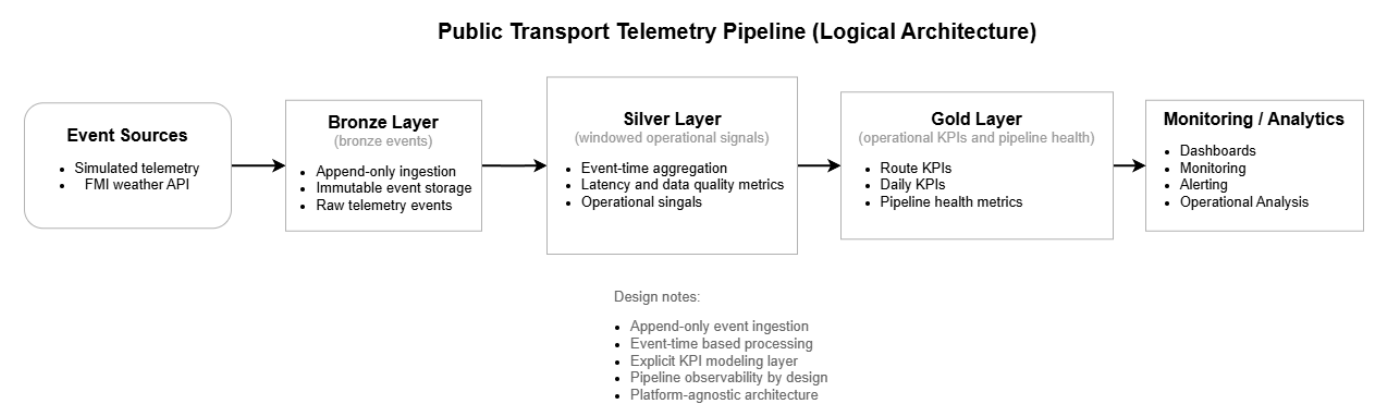
Architecture

The pipeline follows a standard layered structure:

Bronze → Silver → Gold

Each layer has a clear responsibility:

- **Bronze** — raw ingestion (append-only)
- **Silver** — cleaned and aggregated data
- **Gold** — final outputs for reporting and monitoring



Dashboard preview

Pipeline Overview

Recent operational snapshot showing ingestion delay trends and pipeline health metrics.



Route Performance

Route-level KPIs and daily summaries derived from aggregated Gold-layer outputs.



Map View

Route geometry, sampled vehicle points, and optional weather context (FMI).



Deployment and data flow

The project includes a simple but complete serving pipeline:

1. Pipeline runs via GitHub Actions
2. Gold outputs are exported as parquet files
3. Files are uploaded to Azure Blob Storage (using service credentials via GitHub Secrets)
4. Streamlit dashboard (Render) reads these parquet files

Flow:

GitHub Actions → export_gold → Azure Blob → Streamlit (Render)

This avoids direct database connections and keeps the system stable and low-maintenance.

Key design decisions

This project intentionally avoids unnecessary complexity.

- **Append-only ingestion**

Keeps data flow predictable and avoids mutation logic.
Makes debugging and backtracking easier.

- **Unified event model**

All data (telemetry + weather) is normalized into a single schema.
Simplifies downstream processing but requires disciplined structure.

- **Micro-batch processing**

Chosen instead of streaming to reduce infrastructure overhead.
Keeps logic simple while preserving a migration path.

- **Precomputed outputs (parquet)**

Dashboard reads static parquet instead of live queries.
This improves reliability and avoids runtime dependencies.

- **Presentation-layer fallback**

Instead of forcing strict upstream joins, mismatches (e.g. weather vs transit windows) are handled in the UI layer.
This keeps pipeline logic simple and robust.

- **Lightweight orchestration**

GitHub Actions is used instead of Airflow/ADF to keep the system minimal while still automated. This approach was chosen to reduce operational overhead for a small-scale pipeline.

Handling imperfect data

In real-world pipelines, data sources rarely align perfectly.

Examples in this project:

- telemetry and weather events arrive at different times
- some time windows contain only one data source

Instead of forcing strict alignment upstream, this project handles it at the presentation layer.

This keeps the pipeline logic simple while still providing usable outputs.

Data sources

- **Simulated telemetry data**

Represents transport signals such as delay and occupancy.

- **FMI Weather API**

Provides real observational weather data.

Telemetry is simulated intentionally to focus on pipeline design rather than data collection.

Event model

All inputs are transformed into a shared structure.

Core fields:

- `event_time` — when the event occurred
- `ingest_time` — when the event was processed
- `source` — telemetry or weather
- `metric` — type of measurement
- `value` — numeric value
- `unit` — measurement unit
- `attrs` — flexible metadata (e.g. `route_id`)

This enables consistent aggregation and simple schema evolution.

Data layers

Bronze

- append-only raw events
- minimal transformation
- acts as the system of record

Silver

- window-based aggregation (event time)
- route-level and time-based metrics
- basic validation (nulls, duplicates, counts)

Gold

- final metrics for dashboards
- route KPIs and daily summaries
- pipeline health indicators

Outputs are exported as parquet files for stable downstream use.

Dashboard

The Streamlit dashboard provides three views:

- **Pipeline Overview**

Recent pipeline metrics and ingestion delay trends

- **Route Performance**

Route-level KPIs and daily summaries
Includes a table view for inspection

- **Map View**

Route geometry, vehicle points, and optional weather context

The dashboard is designed for clarity and stability rather than heavy interactivity.

Scheduling

Pipeline execution is handled using GitHub Actions.

Note: Execution timing is best-effort and may be slightly delayed depending on GitHub runner availability.

Workflows include:

- `telemetry-refresh.yml` — pipeline execution
- `keepalive_telemetry.yml` — dashboard keepalive
- `keepalive_nyc.yml` — legacy project keepalive

Location:

```
.github/workflows/
```

This setup keeps the system automated without introducing heavy orchestration tools.

Repository structure

```
.github/workflows/
  telemetry-refresh.yml      # scheduled pipeline runs
  keepalive_telemetry.yml    # keep Render service awake

Projects/Public-Transport-Telemetry-Pipeline/

  data/
    bronze/
      bronze_events.csv      # raw append-only events
    silver/
      silver_transit_metrics.csv # aggregated transit metrics
    gold/
      hsl/                   # route geometry and map data
      weather/               # weather observations
    output/
      gold_route_daily.parquet # dashboard-ready daily KPIs
```

```
    gold_route_window.parquet    # window-level KPIs
    pipeline_metrics.parquet    # pipeline health metrics
external/gtfs_hsl/              # GTFS reference data

src/pipeline/
  bronze.py                    # ingestion logic
  silver.py                   # aggregation logic
  gold.py                     # KPI generation
  hsl.py                     # route/map processing
  config.py                   # configuration
  setup.py                    # Spark setup

scripts/
  run_pipeline.py             # main pipeline runner
  export_gold.py              # export parquet outputs
  upload_outputs_to_blob.py   # upload to Azure Blob

streamlit_app/
  Home.py
  pages/
    1_Pipeline_Overview.py
    2_Route_Performance.py
    3_Map_View.py
  utils/
    load_data.py              # parquet loading
    data_access.py            # data access layer
    maps.py                   # map rendering logic

tests/tools/
  inspect_*.py                # inspection scripts
  test_*.py                   # basic checks

notebooks/mvp/                # development notebooks
docs/
  architecture.png

README.md
requirements.txt
.gitignore
```

How to run

Run full pipeline:

```
python scripts/run_pipeline.py --layer full
```

Run individual layers:

```
python scripts/run_pipeline.py --layer bronze
python scripts/run_pipeline.py --layer silver
python scripts/run_pipeline.py --layer gold
```

Inspect outputs:

```
python tests/tools/inspect_gold.py
```

Scope

This project focuses on:

- data integration
- SQL-style aggregation logic
- pipeline reliability
- simple monitoring outputs

Out of scope:

- real-time streaming
- large-scale infrastructure
- predictive modeling

Notes

This project is designed to be:

- easy to understand
- easy to extend
- suitable for real-world adaptation

It can be migrated to cloud platforms such as Azure if needed, but currently avoids unnecessary complexity.

License

This project is covered by the MIT License at the repository root.